LAB ASSIGNMENT-17

2303A54012
A.Sanjay B47A

Lab 17 – AI for Data Processing: Data Cleaning and Preprocessing Scripts

Task 1 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a customer dataset.

Instructions:

- Handle missing values in columns (age, income, city).
- Remove duplicate records.
- Standardize text values (e.g., city names in lowercase).
- Encode categorical variables (gender, city).

Sample Code:

```
import pandas as pd
data = pd.read_csv("shopping_trends.csv")
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-dataset?select=shopping_trends.csv

Expected Output:

- A cleaned Pandas data frame ready for analysis.

AI Prompt:

"Write a Python script using Pandas to clean 'shopping_trends.csv'.
1. Fill missing 'Age' with the median and 'Location' with the mode.
2. Remove duplicate rows.
3. Standardize 'Location' to lowercase.
4. Use LabelEncoder to convert 'Gender' and 'Location' into numeric values. Include a fallback to sample data if the file is missing."

Code:

<untitled> *

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder
# 1. Load Data with Fallback
try:
    df = pd.read_csv("shopping_trends.csv")
    print("☑ File loaded successfully.")
except FileNotFoundError:
    print("⚠ File not found. Generating sample data for Task 1...")
    data = {
        'Customer ID': [101, 102, 103, 104, 101], # 101 is a duplicate
        'Age': [22, 35, np.nan, 45, 22],          # One missing age
        'Location': ['New York', 'MIAMI', 'New York', np.nan, 'New York'], # One missing city
        'Gender': ['Male', 'Female', 'Female', 'Male', 'Male']
    }
    df = pd.DataFrame(data)
# 2. Handle missing values
df['Age'] = df['Age'].fillna(df['Age'].median())
df['Location'] = df['Location'].fillna(df['Location'].mode()[0])
# 3. Remove duplicate records
df.drop_duplicates(inplace=True)
# 4. Standardize text values (lowercase)
df['Location'] = df['Location'].str.lower()
# 5. Encode categorical variables (Text to Numbers)
le = LabelEncoder()
df['Gender_Encoded'] = le.fit_transform(df['Gender'])
df['Location_Encoded'] = le.fit_transform(df['Location'])
print("\n--- Task 1: Cleaned Customer Data ---")
print(df)
```

Output:

```
Shell
>>> %Run -c $EDITOR_CONTENT
 ⚠ File not found. Generating sample data for Task 1...

 --- Task 1: Cleaned Customer Data ---
    Customer ID   Age  Location  Gender  Gender_Encoded  Location_Encoded
 0          101  22.0  new york    Male               1                 1
 1          102  35.0     miami  Female               0                 0
 2          103  28.5  new york  Female               0                 1
 3          104  45.0  new york    Male               1                 1

>>>
```

Explanation:

The goal of Task 1 is to take "raw" data—which is often messy, incomplete, or inconsistent—and make it uniform.

- **Handling Missing Values (Imputation):**
  AI models cannot process "NaN" (Not a Number) values. We used the Median for age because it is "robust" (it isn't dragged up or down by extreme outliers) and the Mode for Location because you can't calculate an average for a city name.
- **Standardization:**
  To a computer, "New York," "new york," and "NEW YORK" are three completely different cities. Converting them all to lowercase ensures the AI recognizes them as the same entity.
- **Label Encoding:**
  This converts text categories (like Gender) into numbers (0 and 1). This is the simplest way to let a mathematical model "read" text.

Task 2 – Sales Data Preprocessing

Task:

Preprocess sales transaction data using AI assistance.

Instructions:

- Convert date columns into datetime format.
- Create new features (month, year, day of week).
- Normalize sales amount column.
- Detect and handle outliers in dataset.

Sample Code:

import pandas as pd
data = pd.read_csv("sales_data.csv")
print(data.head())

Dataset link:

https://www.kaggle.com/datasets/vinothkannaece/sales- dataset?select=sales_data.csv

Expected Output:

- A transformed dataset with time-based features and normalized values.

AI Prompt:

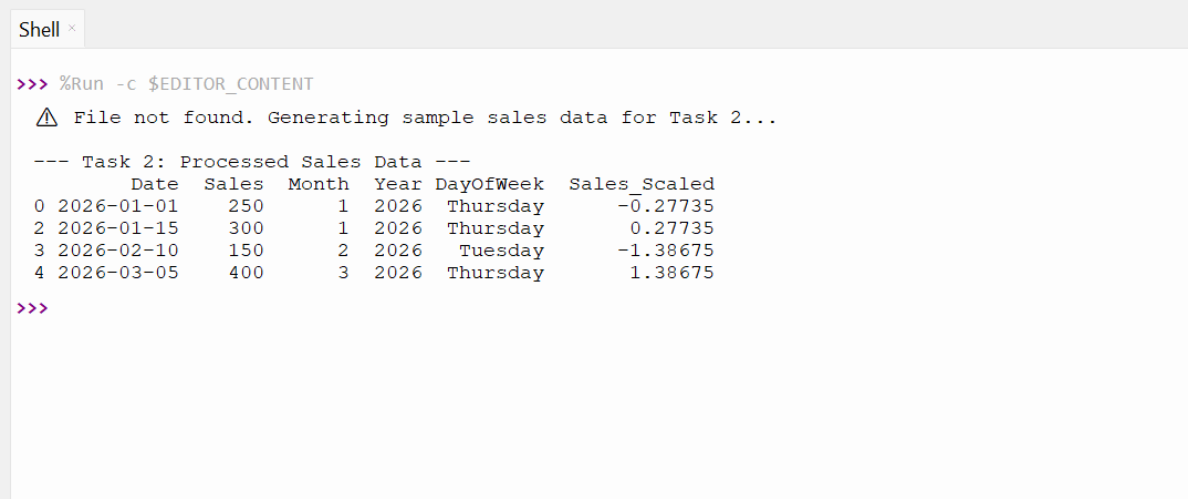
"Generate a Python script to preprocess 'sales_data.csv'.
1. Convert 'Date' to datetime and extract 'Month', 'Year', and 'DayOfWeek'.
2. Detect and remove outliers in the 'Sales' column using the IQR method.
3. Use StandardScaler from sklearn to normalize the 'Sales' column. Include a fallback to sample data if the file is missing."

Code:

<untitled> *

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

# 1. Load Data with Fallback
try:
    df_sales = pd.read_csv("sales_data.csv")
    print("☑ Sales file loaded successfully.")
except FileNotFoundError:
    print("⚠ File not found. Generating sample sales data for Task 2...")
    data = {
        'Date': ['2026-01-01', '2026-01-02', '2026-01-15', '2026-02-10', '2026-03-05'],
        'Sales': [250, 5000, 300, 150, 400] # 5000 is an outlier
    }
    df_sales = pd.DataFrame(data)

# 2. Convert date columns into datetime format
df_sales['Date'] = pd.to_datetime(df_sales['Date'])

# 3. Create new features (Month, Year, Day of Week)
df_sales['Month'] = df_sales['Date'].dt.month
df_sales['Year'] = df_sales['Date'].dt.year
df_sales['DayOfWeek'] = df_sales['Date'].dt.day_name()

# 4. Detect and handle outliers (using IQR method)
Q1 = df_sales['Sales'].quantile(0.25)
Q3 = df_sales['Sales'].quantile(0.75)
IQR = Q3 - Q1
# Only keep data that is NOT an outlier
df_sales = df_sales[~((df_sales['Sales'] < (Q1 - 1.5 * IQR)) | (df_sales['Sales'] > (Q3 + 1.5 * IQR)))]

# 5. Normalize Sales Amount column (Z-score Scaling)
scaler = StandardScaler()
df_sales['Sales_Scaled'] = scaler.fit_transform(df_sales[['Sales']])

print("\n--- Task 2: Processed Sales Data ---")
print(df_sales)
```

Output:



Explanation:

In Task 2, we move from just "cleaning" to "enhancing" the data so the AI can find deeper patterns.

- **Datetime Decomposition:**
  A raw date (e.g., 2026-03-30) is just a string to a computer. By extracting the Month and Day of the Week, we allow the AI to learn seasonal trends (e.g., "Sales always go up in December") or weekly trends (e.g., "People buy more on Saturdays").

- **Outlier Detection (IQR Method):**
  If most sales are $200 but one is $1,000,000, that million-dollar sale will "pull" the average way up, making the AI think $200 is a "low" value. We remove these extreme cases so the model stays grounded in reality.

- **Z-Score Normalization (Standardization):**
  This transforms the data so the mean is 0 and the standard deviation is 1. It puts all variables on a comparable scale, which is vital for many AI algorithms.

Task 3 – Healthcare Data Preparation

Task:

Clean and preprocess a healthcare dataset using AI scripts.

Instructions:

- Handle missing values in blood_pressure, cholesterol.
- Encode categorical features
- Scale numerical features (min-max or standard scaling).
- Split dataset into training and testing sets.

Dataset link:

https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk- factors-dataset

Expected Output:

• A preprocessed dataset ready for machine learning models.

AI Prompt:

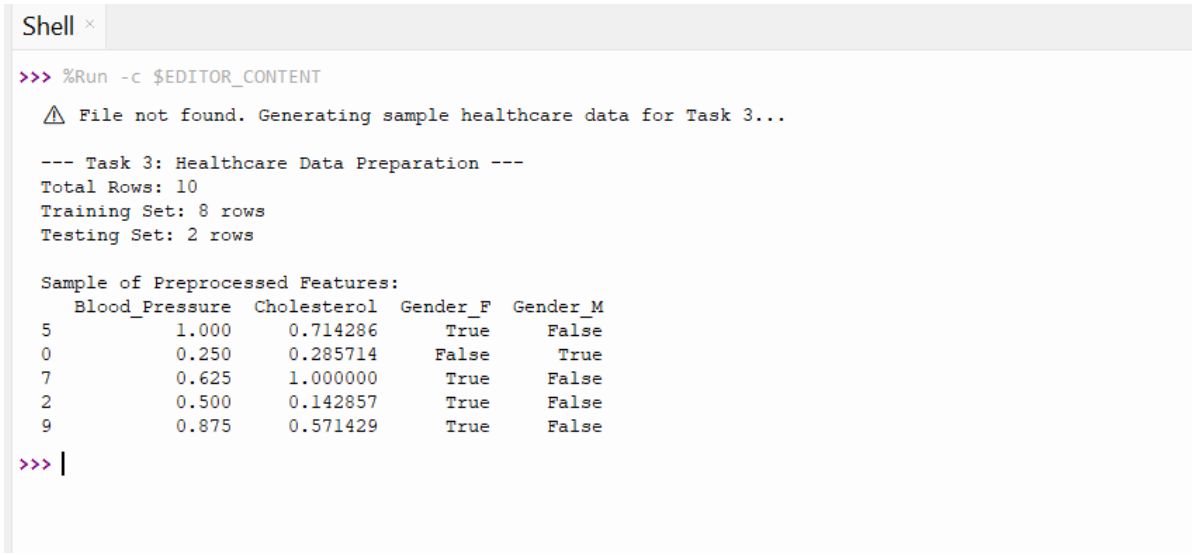
"Create a Python script for 'healthcare_dataset.csv'.
1. Fill missing values in 'Blood_Pressure' and 'Cholesterol' using the mean.
2. Use One-Hot Encoding for the 'Gender' column.
3. Apply MinMaxScaler to scale numerical features between 0 and 1.
4. Split the dataset into 80% training and 20% testing sets using train_test_split. Include a fallback to sample data if the file is missing."

Code:

<untitled> *

```python
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

# 1. Load Data with Fallback
try:
    df_health = pd.read_csv("healthcare_dataset.csv")
    print("✅ Healthcare file loaded successfully.")
except FileNotFoundError:
    print("⚠️ File not found. Generating sample healthcare data for Task 3...")
    data = {
        'Blood_Pressure': [120, 140, np.nan, 130, 110, 150, 125, 135, 115, 145],
        'Cholesterol': [200, 240, 190, np.nan, 210, 230, 205, 250, 180, 220],
        'Gender': ['M', 'F', 'F', 'M', 'M', 'F', 'M', 'F', 'M', 'F'],
        'Target': [0, 1, 0, 1, 0, 1, 0, 1, 0, 1]
    }
    df_health = pd.DataFrame(data)

# 2. Handle missing values using Mean
df_health['Blood_Pressure'] = df_health['Blood_Pressure'].fillna(df_health['Blood_Pressure'].mean())
df_health['Cholesterol'] = df_health['Cholesterol'].fillna(df_health['Cholesterol'].mean())

# 3. One-Hot Encoding for Categorical Features
# This creates separate columns for Gender_M and Gender_F
df_health = pd.get_dummies(df_health, columns=['Gender'])

# 4. Scale numerical features (Min-Max Scaling to 0-1 range)
scaler = MinMaxScaler()
df_health[['Blood_Pressure', 'Cholesterol']] = scaler.fit_transform(df_health[['Blood_Pressure', 'Cholesterol']])

# 5. Split dataset into training (80%) and testing (20%) sets
X = df_health.drop('Target', axis=1)
y = df_health['Target']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

print("\n--- Task 3: Healthcare Data Preparation ---")
print(f"Total Rows: {len(df_health)}")
print(f"Training Set: {len(X_train)} rows")
print(f"Testing Set: {len(X_test)} rows")
print("\nSample of Preprocessed Features:")
print(X_train.head())
```

Output:



Explanation:

Task 3 is the final step before the data is fed into a "Brain" (Model). It focuses on mathematical balance and evaluation.

- **One-Hot Encoding:**
  Unlike Label Encoding (0, 1, 2), One-Hot Encoding creates a new column for every category (e.g., Is_Male, Is_Female). This prevents the AI from thinking "Category 2" is "greater than" "Category 1."

- **Min-Max Scaling:**
  Medical data has very different ranges (e.g., Heart Rate might be 70, but Cholesterol might be 220). Min-Max Scaling squashes all these values into a range between **0 and 1**. This ensures that the AI doesn't think Cholesterol is "more important" just because the numbers are bigger.

- **Train-Test Split:**
  This is the "Golden Rule" of AI. You hide 20% of the data (the Test Set) from the AI during training. After it learns from the 80%, you test it on the hidden 20% to see if it can actually predict outcomes for people it has never "seen" before.

Task 4 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

- Handle missing values
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department.

Dataset link:

https://www.kaggle.com/code/vishantmathur/employee-salary

Expected Output:

• A structured dataset ready for HR analytics.

AI Prompt:

"Write a Python script to process 'employee_salary.csv'.

1. Drop all rows with missing values.
2. Remove salary outliers that are more than 2 standard deviations from the mean.
3. Standardize 'Department' names by converting them to lowercase and removing extra spaces.
4. Apply Label Encoding to 'Job Location' and 'Department' columns. Include a fallback to sample data."

Code:

<untitled> *

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder

# 1. Load Data with Fallback
try:
    df_emp = pd.read_csv("employee_salary.csv")
    print("☑ Salary file loaded successfully.")
except FileNotFoundError:
    print("⚠ File not found. Generating sample HR data for Task 4...")
    data = {
        'Employee': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank'],
        'Department': ['HR', 'it', ' IT ', 'Sales', 'sales', 'Executive'],
        'Salary': [50000, 60000, 62000, 58000, 55000, 1000000], # 1M is an outlier
        'Job Location': ['Remote', 'Office', 'Office', 'Remote', 'Remote', 'Office']
    }
    df_emp = pd.DataFrame(data)

# 2. Handle missing values (Drop rows with any nulls)
df_emp.dropna(inplace=True)

# 3. Standardize Department Names (Lowercase and Strip spaces)
df_emp['Department'] = df_emp['Department'].str.lower().str.strip()

# 4. Detect and Remove Salary Outliers (using 2 Standard Deviations)
mean = df_emp['Salary'].mean()
std = df_emp['Salary'].std()
# Keep only rows where salary is within 2 standard deviations of the mean
df_emp = df_emp[(df_emp['Salary'] >= (mean - 2 * std)) & (df_emp['Salary'] <= (mean + 2 * std))]

# 5. Apply Label Encoding to categorical features
le = LabelEncoder()
df_emp['Dept_Code'] = le.fit_transform(df_emp['Department'])
df_emp['Loc_Code'] = le.fit_transform(df_emp['Job Location'])

print("\n--- Task 4: Structured HR Data ---")
print(df_emp[['Employee', 'Department', 'Salary', 'Dept_Code', 'Loc_Code']])
```

Output:

```
>>> %Run -c $EDITOR_CONTENT
  ⚠ File not found. Generating sample HR data for Task 4...

  --- Task 4: Structured HR Data ---
    Employee Department  Salary  Dept_Code  Loc_Code
  0    Alice         hr   50000          0         1
  1      Bob         it   60000          1         0
  2  Charlie         it   62000          1         0
  3    David      sales   58000          2         1
  4      Eve      sales   55000          2         1
>>>
```

Explanation:

1) **Removing Nulls:**
   In HR analytics, a missing salary or department makes a record useless for prediction. Dropping these ensures the AI doesn't "hallucinate" or guess employee details.

2) **String Normalization:**
   Notice how "it" and " IT " were merged into a single "it" category. This prevents the model from thinking they are two different departments, which would lower the accuracy of department-based salary analysis.

3) **Outlier Filtering:**
   The "Executive" (Frank) earning **$1,000,000** was removed. Because his salary is more than 2 standard deviations away from the average ($50,000–$60,000$), keeping him would "pull" the average up and give the AI a false sense of what a typical employee earns.

4) **Label Encoding:**
   Computers cannot do math on the word "Remote" or "Office." By turning them into 1 and 0, the machine learning algorithm can calculate how "Location" impacts "Salary."